CODE :-

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char item)
{
   if (top == MAX - 1)
   {
      printf("Stack Overflow\n");
      return;
   }

   top++;
   stack[top] = item;
}

char pop()
{
   if (top == -1)
   {
      return '\0';
   }

   return stack[top--];
```

```
}

char peek()
{
   if (top == -1)
   {
      return '\0';
   }

   return stack[top];
}

int isEmpty()
{
   return top == -1;
}

int precedence(char op)
{
   if (op == '^')
      return 3;

   if (op == '*' || op == '/')
      return 2;

   if (op == '+' || op == '-')
      return 1;

   return 0;
}
```

```
void infixToPostfix(char infix[], char postfix[])
{
    int i = 0;
    int j = 0;
    char ch;

    top = -1;

    while (infix[i] != '\0')
    {
        ch = infix[i];

        if (isalnum(ch))
        {
            postfix[j++] = ch;
        }
        else if (ch == '(')
        {
            push(ch);
        }
        else if (ch == ')')
        {
            while (!isEmpty() && peek() != '(')
            {
                postfix[j++] = pop();
            }

            if (!isEmpty())
                pop();
        }
        else
```

```
        {
            while (!isEmpty() &&
                   precedence(peek()) >= precedence(ch))
            {
                postfix[j++] = pop();
            }

            push(ch);
        }

        i++;
    }

    while (!isEmpty())
    {
        postfix[j++] = pop();
    }

    postfix[j] = '\0';
}

void postfixToInfix(char postfix[], char infix[])
{
    char stackExp[MAX][MAX];
    int topExp = -1;
    int i;
    char ch;
    char op1[MAX];
    char op2[MAX];

    for (i = 0; postfix[i] != '\0'; i++)
```

```
    {
        ch = postfix[i];

        if (isalnum(ch))
        {
            topExp++;
            stackExp[topExp][0] = ch;
            stackExp[topExp][1] = '\0';
        }
        else
        {
            strcpy(op2, stackExp[topExp--]);
            strcpy(op1, stackExp[topExp--]);

            topExp++;

            snprintf(stackExp[topExp],
                    MAX,
                    "(%s%c%s)",
                    op1,
                    ch,
                    op2);
        }
    }

    strcpy(infix, stackExp[topExp]);
}

void displayStack()
{
    int i;
```

```
    if (top == -1)
    {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack elements: ");

    for (i = top; i >= 0; i--)
    {
        printf("%c ", stack[i]);
    }

    printf("\n");
}

int main()
{
    int choice;
    char expression[MAX];
    char result[MAX];

    do
    {
        printf("\n EXPRESSION CONVERSION USING STACK  \n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Infix to Postfix\n");
        printf("4. Postfix to Infix\n");
        printf("5. Display Stack\n");
```

```
printf("6. Exit\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
   case 1:
   {
      char item;

      printf("Enter character to push: ");
      scanf(" %c", &item);

      push(item);

      printf("%c pushed into stack.\n", item);
      break;
   }

   case 2:
   {
      char item = pop();

      if (item == '\0')
         printf("Stack Underflow\n");
      else
         printf("Popped element: %c\n", item);

      break;
   }
```

```
    case 3:
        printf("Enter infix expression: ");
        scanf("%s", expression);

        infixToPostfix(expression, result);

        printf("Postfix expression: %s\n", result);
        break;

    case 4:
        printf("Enter postfix expression: ");
        scanf("%s", expression);

        postfixToInfix(expression, result);

        printf("Infix expression: %s\n", result);
        break;

    case 5:
        displayStack();
        break;

    case 6:
        printf("Exiting program...\n");
        break;

    default:
        printf("Invalid choice!\n");
}
```

```
    } while (choice != 6);

    return 0;
}
```

Output :-

Test 1 — Infix to Postfix

EXPRESSION CONVERSION USING STACK

1. Push

2. Pop

3. Infix to Postfix

4. Postfix to Infix

5. Display Stack

6. Exit

Enter your choice: 3

Enter infix expression: A+B*C

Postfix expression: ABC*+

Test 2 — Infix to Postfix with Parentheses

Enter your choice: 3

Enter infix expression: (A+B)*C

Postfix expression: AB+C*

Test 3 — Postfix to Infix

Enter your choice: 4

Enter postfix expression: ABC*+

Infix expression: (A+(B*C))

Test 4 — Push and Pop

Enter your choice: 1

Enter character to push: A

A pushed into stack.

Enter your choice: 1

Enter character to push: B

B pushed into stack.

Enter your choice: 5

Stack elements: B A

Enter your choice: 2

Popped element: B

Test 5 — Stack Underflow

Enter your choice: 2

Stack Underflow

WRITEUP :-

Assignment no:- 7

Problem Statement :-
Implement stacks as an ADT and apply it for different expression conversions.

Objectives :-
1. To study : Stack and its operations
2. To study the importance of expression conversions

Theory :-
Stack : Special case of ordered list also called as restricted / controlled list where insertion and deletion happen at only one end, called as top of stack. Elements are added to and removed from the top of the stack. The last element to be added is the first to be removed. Only access to the stack is top element

Basic Stack operations :-
- isEmpty() - checking stack is Empty
- isFull() :- Checking stack is full
- Push() :- Pushing Element on top of stack.
- Pop() :- Popping top element from stack

Application of Stack :-
- Expression evaluation
- Infix to Postfix & Prefix conversion
- Function call management
- Recursion
- Parentheses matching
- Undo / Redo operations in editors
- Browser back and forward navigation
- Depth-First Search (DFS) in graphs
- Syntax parsing in Compilers
- String reversal

Expression conversion is the process of converting an arithmetic expression from one notation to another using a stack. The three notations are :
- Infix :- operator is btw operands.
- Prefix :- operator is before operands.
- Postfix :- Operator is after operands.

A stack is used to temporarily store operators and parentheses during the conversion.

Implementation :-
- Platform :-
  64-bit Open Source Linux or its derivatives
  Open Source C Programming tool like gcc / eclipse with
- Test cond" :-

1) Input infix expression and check the postfix or Prefix output
2) Input postfix or prefix expression and check its infix, prefix or postfix.

Pseudo Code :-

A) Push :
PUSH (item)
1. If TOP == MAX-1
   Print "Stack Overflow"
   Return
2. TOP = TOP + 1
3. STACK[TOP] = Item
4. Stop

B) POP
POP()
1. If TOP == -1
   Print "Stack Underflow"
   Return
2. Item = STACK[TOP]
3. TOP = TOP - 1
4. Return Item

C) Infix to Postfix
Infix_TO_POSTFIX (expression)
1. Initialize an empty stack
2. Scan the expression from left to right.
3. If the character is an operand
   Add it to postfix expression.
4. If the character is '('
   PUSH('(') onto stack.
5. If the character is ')'
   POP operators and add them to postfix
   until '(' is found.
   Remove '(' from stack.
6. If the character is an operator :
   While stack is not empty AND
   precedence (stop) >= precedence (current operator)
   POP from stack and add to postfix
   PUSH current operator onto stack.
7. After scanning the complete expression
   POP all remaining operators
   and add them to Postfix.
8. Display postfix expression
9. Stop

D) Postfix to Infix :
POSTFIX_TO_INFIX (expression)
1. Initialize an empty stack
2. Scan the postfix expression from left to right.
3. If the character is an operand :
   PUSH it onto the stack
4. If the character is an operator :
   operand2 = POP()
   operand1 = POP()

   expression = '(' + operand1 +
   operator +
   operand2 + ')'

   PUSH expression onto stack.

5. After scanning the complete expression:
   POP the final expression
6. Display the infix expression.
7. Stop

Time Complexity :-

Operation	Time Complexity
Push	O(1)
Pop	O(1)
IsEmpty	O(1)
IsFull	O(1)
Infix → Postfix	O(n)
Postfix → Infix	O(n)

Conclusion :-
Thus, implemented Stack Operations using using Array concepts. This System is able to apply different expression conversions.

FAQs :-

1) What is the advantage of prefix and postfix over infix expression?
→ Prefix and postfix expressions are easier for computers to evaluate because they do not require parentheses and operator-precedence rules during evaluation. They provide an unambiguous order of operations.

2) Why is stack used in expression conversion?
→ It is used to temporarily store operators and parentheses during expression conversion. It helps arrange the operators in the correct order according to their precedence.

3) Give stack full and stack empty cond" for stack
→ • Stack full : When top == MAX-1, the stack is full. A further push operation causes overflow.
• Stack Empty : When top == -1, the stack is empty. A pop operation causes stack underflow.

R
5/11